Schritt 37 — GraphQL Directives

Nach Fragments (35) und Aliases (36) folgt in Schritt 37 Directives.

Die wichtigste Frage zuerst:

> Gehört eine Directive zu GraphQL oder zu Rails? Sendet der Client die Directive im Request?

Eine Directive ist GraphQL-Syntax. Bei den zwei wichtigsten Standard-Directives @include und @skip schreibt der Client sie direkt in das GraphQL-Dokument, das an /graphql gesendet wird. Rails benötigt dafür keinen eigenen Resolver.

1. Was ist eine Directive?

Eine Directive kann beeinflussen, wie GraphQL einen Teil einer Operation ausführt.

Syntax:

```
@directiveName(argument: value)
```

Die zwei wichtigsten Directives:

```
@include(if: Boolean!)
@skip(if: Boolean!)
```

Einfach erklärt:

```
@include
→ Field nehmen, wenn condition = true

@skip
→ Field überspringen, wenn condition = true
```

2. @include

Angenommen, Project hat:

```
id
name
description
tasks
```

Manchmal benötigt das Frontend nur:

```
id
name
```

Manchmal zusätzlich:

```
tasks
```

Dann können wir schreiben:

```
query GetProject(
  $id: ID!
  $includeTasks: Boolean!
) {
  project(id: $id) {
    id
    name

    tasks @include(if: $includeTasks) {
      nodes {
        id
        title
      }
    }
  }
}
```

Wichtig ist:

```
@include(if: $includeTasks)
```

3. Variables für @include

Der Client sendet:

```
{
  "id": "10",
```

```
  "includeTasks": true
}
```

Wenn:

```
includeTasks = true
```

nimmt GraphQL:

```
tasks
```

Die Response kann so aussehen:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM",
      "tasks": {
        "nodes": [
          {
            "id": "1",
            "title": "Build API"
          }
        ]
      }
    }
  }
}
```

4. Wenn includeTasks = false

Variables:

```
{
  "id": "10",
  "includeTasks": false
}
```

GraphQL lässt Folgendes weg:

```
tasks
```

Response:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM"
    }
  }
}
```

Wichtig:

Nicht:

```
{
  "tasks": null
}
```

Sondern das Field erscheint überhaupt nicht in der Response.

5. Vollständiger Axios Client Request

```
import axios from "axios";

const query = `
  query GetProject(
    $id: ID!
    $includeTasks: Boolean!
  ) {
    project(id: $id) {
      id
      name

      tasks @include(if: $includeTasks) {
        nodes {
          id
          title
```

```
          status
        }
      }
    }
  }
`;

const response = await axios.post(
  "/graphql",
  {
    query,

    variables: {
      id: "10",
      includeTasks: true
    },

    operationName: "GetProject"
  },
  {
    withCredentials: true
  }
);
```

Die Directive befindet sich in:

```
query
```

Die Variable:

```
includeTasks: true
```

befindet sich in:

```
variables
```

6. Was erhält Rails?

Der GraphqlController bleibt ganz normal:

```
def execute
  result = MyAppSchema.execute(
    params[:query],
    variables: prepare_variables(params[:variables]),
    context: {
      current_user: current_user
    },
    operation_name: params[:operationName]
  )

  render json: result
end
```

Du brauchst nicht:

```
def include_tasks
end
```

Und auch nicht:

```
class IncludeDirective
end
```

für das Standard-@include.

Die GraphQL Engine verarbeitet es.

7. @skip

@skip funktioniert umgekehrt zu @include.

Beispiel:

```
query GetProject(
  $id: ID!
  $skipDescription: Boolean!
) {
  project(id: $id) {
    id
    name
```

```
    description
      @skip(if: $skipDescription)
  }
}
```

Wenn:

```
{
  "id": "10",
  "skipDescription": true
}
```

dann:

```
description
→ wird übersprungen
```

Response:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM"
    }
  }
}
```

8. Wenn $skipDescription = false

Variables:

```
{
  "id": "10",
  "skipDescription": false
}
```

Response:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM",
      "description": "Internal CRM project"
    }
  }
}
```

Formel:

```
@skip(if: true)
→ überspringen

@skip(if: false)
→ nehmen
```

Dagegen:

```
@include(if: true)
→ nehmen

@include(if: false)
→ weglassen
```

9. Merktabelle

```
| Directive | Condition | Ergebnis |
|---|---:|---|
| `@include` | `true` | Field nehmen |
| `@include` | `false` | Field weglassen |
| `@skip` | `true` | Field weglassen |
| `@skip` | `false` | Field nehmen |
```

Das ist der wichtigste Teil zum Merken.

10. Directive + Fragment

Hier werden Schritt 35 + Schritt 37 kombiniert.

Fragment:

```
fragment TaskFields on Task {
  id
  title
  status
  priority
}
```

Query:

```
query GetProject(
  $id: ID!
  $includeTasks: Boolean!
) {
  project(id: $id) {
    id
    name

    tasks @include(if: $includeTasks) {
      nodes {
        ...TaskFields
      }
    }
  }
}
```

Hier:

```
...TaskFields
→ Fragment

@include
→ Directive
```

11. Directive auf einem Fragment Spread

Nicht nur auf Fields.

Du kannst schreiben:

```
query GetProject(
  $id: ID!
  $includeDetails: Boolean!
) {
  project(id: $id) {
    id
    name

    ...ProjectDetails
      @include(if: $includeDetails)
  }
}

fragment ProjectDetails on Project {
  description
  status
  createdAt
  updatedAt
}
```

Wenn:

```
{
  "id": "10",
  "includeDetails": false
}
```

wird das gesamte:

```
...ProjectDetails
```

weggelassen.

12. Directive + Alias + Fragment

Jetzt kombinieren wir Schritt 35 + 36 + 37:

```
fragment TaskSummaryFields on Task {
  id
  title
  status
  priority
}

query GetTaskBoard(
  $projectId: ID!
  $includeDone: Boolean!
) {
  todoTasks: tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
  }

  doneTasks: tasks(
    projectId: $projectId
    filter: {
      status: [DONE]
    }
    first: 10
  ) @include(if: $includeDone) {
    nodes {
      ...TaskSummaryFields
    }
  }
}
```

Hier:

```
GetTaskBoard
→ Operation Name

$projectId
$includeDone
→ Variables

todoTasks
doneTasks
→ Aliases

...TaskSummaryFields
→ Fragment

@include
→ Directive
```

Das ist ein realistischer GraphQL Request.

13. Axios für den obigen Request

```
const response = await axios.post(
  "/graphql",
  {
    query: `
      fragment TaskSummaryFields on Task {
        id
        title
        status
        priority
      }

      query GetTaskBoard(
        $projectId: ID!
        $includeDone: Boolean!
      ) {
        todoTasks: tasks(
          projectId: $projectId
          filter: {
            status: [TODO]
```

```
          }
          first: 10
        ) {
          nodes {
            ...TaskSummaryFields
          }
        }

        doneTasks: tasks(
          projectId: $projectId
          filter: {
            status: [DONE]
          }
          first: 10
        ) @include(if: $includeDone) {
          nodes {
            ...TaskSummaryFields
          }
        }
      }
    `,

    variables: {
      projectId: "10",
      includeDone: true
    },

    operationName: "GetTaskBoard"
  },
  {
    withCredentials: true
  }
);
```

14. Warum nicht mit JavaScript lösen?

Du könntest denken:

```
if (includeTasks) {
  // Query mit tasks
} else {
  // Query ohne tasks
}
```

Danach zwei Queries erstellen:

```
GET_PROJECT
```

und:

```
GET_PROJECT_WITH_TASKS
```

Das ist möglich. Wenn sich aber nur einige Fields unterscheiden, ist eine Directive oft sauberer:

```
tasks @include(if: $includeTasks)
```

Eine Operation:

```
GetProject
```

eine Variable:

```
includeTasks
```

ist einfacher zu verwalten.

15. Reduziert eine Directive die Response-Daten?

Ja.

Wenn:

```
tasks @include(if: false)
```

wird tasks nicht in das Result aufgenommen.

Das kann helfen:

```
kleinere Response
weniger Fields zu resolven
weniger unnötige Arbeit
```

Aber wichtig:

> Eine Directive ist kein Database Optimization Tool.

16. Reduziert eine Directive SQL?

Sie kann indirekt einen Resolver und die dazugehörige SQL-Abfrage vermeiden, wenn ein Field ausgelassen wird.

Beispiel:

```
tasks @include(if: false)
```

GraphQL muss tasks nicht resolven.

Wenn der tasks Resolver normalerweise ausführt:

```
project.tasks
```

kann diese Query dadurch entfallen.

Aber die Schlussfolgerung:

```
directive
=
SQL optimizer
```

ist falsch.

N+1 muss weiterhin behandelt werden mit:

```
Dataloader
preload
includes
batch loading
```

17. Directive für Authorization?

Nein.

Das wäre ein gefährlicher Fehler.

Nicht:

```
adminData
  @include(if: $isAdmin)
```

und dann glauben:

```
{
  "isAdmin": false
}
```

würde die Daten schützen.

Der Client kann einfach senden:

```
{
  "isAdmin": true
}
```

Deshalb sind:

```
@include
@skip
```

keine Authorization.

18. Authorization muss in Rails liegen

Beispiel:

```
def resolve(project_id:, **)
  project =
    Project.find(project_id)

  authorize project, :show?

  project
end
```

Oder Field-Authorization passend zu deiner Architektur.

Die vertrauenswürdige Quelle ist:

```
context[:current_user]
```

und kommt von:

```
current_user
```

aus Devise.

Nicht:

```
$isAdmin
```

vom Client.

19. Falsch und richtig bei Security

Falsch

```
query GetProject(
  $isAdmin: Boolean!
) {
  project(id: "10") {
    secretData
      @include(if: $isAdmin)
  }
}
```

Client:

```
{
  "isAdmin": true
}
```

Der Client kann den Boolean selbst bestimmen.

Das ist keine Security.

Richtig

Rails:

```
def secret_data
  user =
    context[:current_user]

  raise GraphQL::ExecutionError,
        "Forbidden" unless user.admin?

  object.secret_data
end
```

Die Directive entscheidet nur:

```
möchte der Client dieses Field anfragen?
```

Die Policy entscheidet:

```
darf der Client dieses Field erhalten?
```

20. Directive und React UI

Angenommen, die UI hat eine Checkbox:

```
<label>
  <input
    type="checkbox"
    checked={includeDone}
    onChange={(event) =>
      setIncludeDone(event.target.checked)
    }
  />

  Show done tasks
</label>
```

State:

```
const [
  includeDone,
  setIncludeDone
```

```
] = useState(false);
```

Request:

```
variables: {
  projectId: "10",
  includeDone
}
```

GraphQL:

```
doneTasks
  @include(if: $includeDone)
```

Das ist ein sehr klarer Use Case für eine Directive.

21. Query String nicht dynamisch bauen

Nicht empfohlen:

```
let fields = `
  id
  name
`;

if (includeTasks) {
  fields += `
    tasks {
      nodes {
        id
      }
    }
  `;
}
```

und danach:

```
const query = `
  query {
    project {
      ${fields}
    }
  }
`;
```

Das erschwert:

```
maintain
test
type check
persisted query
operation tracking
```

Besser:

```
tasks
  @include(if: $includeTasks)
```

22. Directive mit Boolean Default Value

Du kannst deklarieren:

```
query GetProject(
  $id: ID!
  $includeTasks: Boolean! = false
) {
  project(id: $id) {
    id
    name

    tasks @include(if: $includeTasks) {
      nodes {
        id
        title
      }
    }
  }
}
```

Wenn der Client nicht sendet:

```
{
  "includeTasks": ...
}
```

gilt:

```
includeTasks = false
```

23. Axios mit Default Value

Der Client braucht nur:

```
variables: {
  id: "10"
}
```

Nicht:

```
includeTasks: false
```

weil die Operation bereits hat:

```
$includeTasks: Boolean! = false
```

24. Gibt es Custom Directives?

Ja.

Neben:

```
@include
@skip
```

kann eine GraphQL-Implementierung Custom Directives unterstützen.

Konzeptionelle Beispiele:

```
@auth
@deprecated
@upper
@cache
```

Aber man muss unterscheiden.

@include und @skip:

```
GraphQL standard executable directives
```

Custom Directive:

```
wird von Application/Server selbst definiert
```

25. @deprecated ist ebenfalls eine Directive

Beispiel im Schema:

```
type Project {
  oldName: String
    @deprecated(
      reason: "Use name instead"
    )

  name: String!
}
```

In graphql-ruby deklarierst du die Deprecation normalerweise über die Field-API, zum Beispiel:

```
field :old_name,
      String,
      null: true,
      deprecation_reason: "Use name instead"
```

Das führt direkt zu:

Schritt 43 — Schema Evolution & Deprecation.

26. Executable Directive vs Schema Directive

Diese zwei Konzepte solltest du unterscheiden.

Executable

Wird vom Client in einer Operation verwendet:

```
@include
@skip
```

Beispiel:

```
tasks
  @include(if: $includeTasks)
```

Schema-related Directive

Beschreibt Verhalten oder Metadaten im Schema:

```
@deprecated
```

Beispiel:

```
oldName
  @deprecated(reason: "Use name")
```

Nicht jede Directive hat also denselben Zweck.

27. Directive + Pagination

Project Detail kann Tasks optional laden:

```
query GetProject(
  $id: ID!
  $includeTasks: Boolean!
  $first: Int!
  $after: String
) {
  project(id: $id) {
    id
    name

    tasks(
      first: $first
      after: $after
    ) @include(if: $includeTasks) {
      nodes {
        id
        title
      }

      pageInfo {
        hasNextPage
        endCursor
      }
    }
  }
}
```

Hier:

```
$first / $after
→ Pagination

$includeTasks
→ Directive condition
```

28. Directive + Filter + Sort + Pagination

Jetzt kombinieren wir die vorherigen Schritte:

```
query GetProjectTasks(
  $projectId: ID!
  $filter: TaskFilterInput
  $sortBy: TaskSortFieldEnum
  $sortDirection: SortDirectionEnum
  $first: Int!
  $after: String
  $includeTasks: Boolean!
) {
  project(id: $projectId) {
    id
    name

    tasks(
```

```
      filter: $filter
      sortBy: $sortBy
      sortDirection: $sortDirection
      first: $first
      after: $after
    ) @include(if: $includeTasks) {
      nodes {
        ...TaskSummaryFields
      }

      pageInfo {
        hasNextPage
        endCursor
      }
    }
  }
}
```

Das ist bereits eine GraphQL Query, die sehr nah an Production ist.

29. Client Variables

```
const variables = {
  projectId: "10",

  filter: {
    status: [
      "TODO",
      "IN_PROGRESS"
    ],

    priority: [
      "HIGH"
    ]
  },

  sortBy: "DUE_DATE",
  sortDirection: "ASC",

  first: 20,
  after: null,

  includeTasks: true
};
```

30. Fragment + Alias + Directive + Variables

Ein Gesamtbeispiel:

```
fragment TaskSummaryFields on Task {
  id
  title
  status
  priority
  dueDate
}

query GetProjectBoard(
  $projectId: ID!
  $includeTodo: Boolean!
  $includeDone: Boolean!
) {
  project(id: $projectId) {
    id
    name
  }

  todoTasks: tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
    first: 10
  ) @include(if: $includeTodo) {
    nodes {
      ...TaskSummaryFields
    }
```

```
  }

  doneTasks: tasks(
    projectId: $projectId
    filter: {
      status: [DONE]
    }
    first: 10
  ) @include(if: $includeDone) {
    nodes {
      ...TaskSummaryFields
    }
  }
}
```

Variables:

```
{
  "projectId": "10",
  "includeTodo": true,
  "includeDone": false
}
```

Response:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM"
    },

    "todoTasks": {
      "nodes": [
        {
          "id": "1",
          "title": "GraphQL API",
          "status": "TODO",
          "priority": "HIGH",
          "dueDate": "2026-08-25"
        }
      ]
    }
  }
}
```

doneTasks erscheint nicht, weil:

```
includeDone = false
```

31. Diagramm zu Schritt 37

```
CLIENT
│
├── Operation
│   GetProjectBoard
│
├── Variables
│   projectId
│   includeTodo
│   includeDone
│
├── Aliases
│   todoTasks
│   doneTasks
│
├── Fragment
│   TaskSummaryFields
│
└── Directives
    @include
        │
        ▼
GraphQL Document
        │
        ▼
POST /graphql
        │
        ▼
```

```
GraphQL Engine
        │
        ├── validate variables
        ├── validate fragments
        ├── evaluate directives
        └── build execution
        │
        ▼
Rails Resolver
        │
        ▼
Policy / Service
        │
        ▼
Database
        │
        ▼
Response
```

32. Vier Begriffe, die leicht verwechselt werden

```
Variable
→ dynamische Daten

Fragment
→ Fields wiederverwenden

Alias
→ Response Key umbenennen

Directive
→ steuert, ob eine Selection
  ausgeführt/eingeschlossen wird
```

Beispiel:

```
query GetBoard(
  $projectId: ID!
  $showDone: Boolean!
) {
  doneTasks: tasks(
    projectId: $projectId
  ) @include(if: $showDone) {
    nodes {
      ...TaskFields
    }
  }
}
```

Analyse:

```
GetBoard
→ Operation Name

$projectId
$showDone
→ Variables

doneTasks
→ Alias

tasks
→ echtes Field im Schema

@include
→ Directive

...TaskFields
→ Fragment
```

Merksätze zu Schritt 37

```
Directive
→ gehört zu GraphQL

Wo wird sie geschrieben?
→ im GraphQL-Dokument des Clients

Ist sie im Axios Request?
```

```
→ ja, innerhalb des query Strings

Braucht Rails einen eigenen Resolver
für @include / @skip?
→ nein

@include(if: true)
→ nehmen

@include(if: false)
→ weglassen

@skip(if: true)
→ weglassen

@skip(if: false)
→ nehmen

Für Authorization?
→ NEIN

Kann sie die Anzahl resolvter Fields reduzieren?
→ ja

Ist sie ein SQL Optimizer?
→ nein
```

Der wichtigste Punkt aus Schritt 37:

```
Variable
       ↓
$includeTasks
       ↓
Directive
       ↓
@include(if: $includeTasks)
       ↓
GraphQL entscheidet,
ob diese Selection ausgeführt wird
```

Und von Schritt 33 → 37 hast du jetzt die Kette der GraphQL-Client-Konzepte:

```
33 Variables
    ↓
34 Operation Name
    ↓
35 Fragments
    ↓
36 Aliases
    ↓
37 Directives
```

Alles wird im GraphQL-Dokument kombiniert, das das Frontend an Rails /graphql sendet.